

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины «Искусственный интеллект и машинное обучение

Вариант 20

Выполнил:
Скоробогатов Виктор Андреевич
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Ход работы

Ссылка на репозиторий:

<https://github.com/LostsSs78/AI-Lab4>

Для выполнения работы был создан новый общедоступный репозиторий на GitHub, использующий лицензию MIT и язык программирования Python, далее он был клонирован на персональный компьютер, также был дополнен файл .gitignore необходимыми правилами для языка Python, IDE PyCharm и Jupyter Lab:

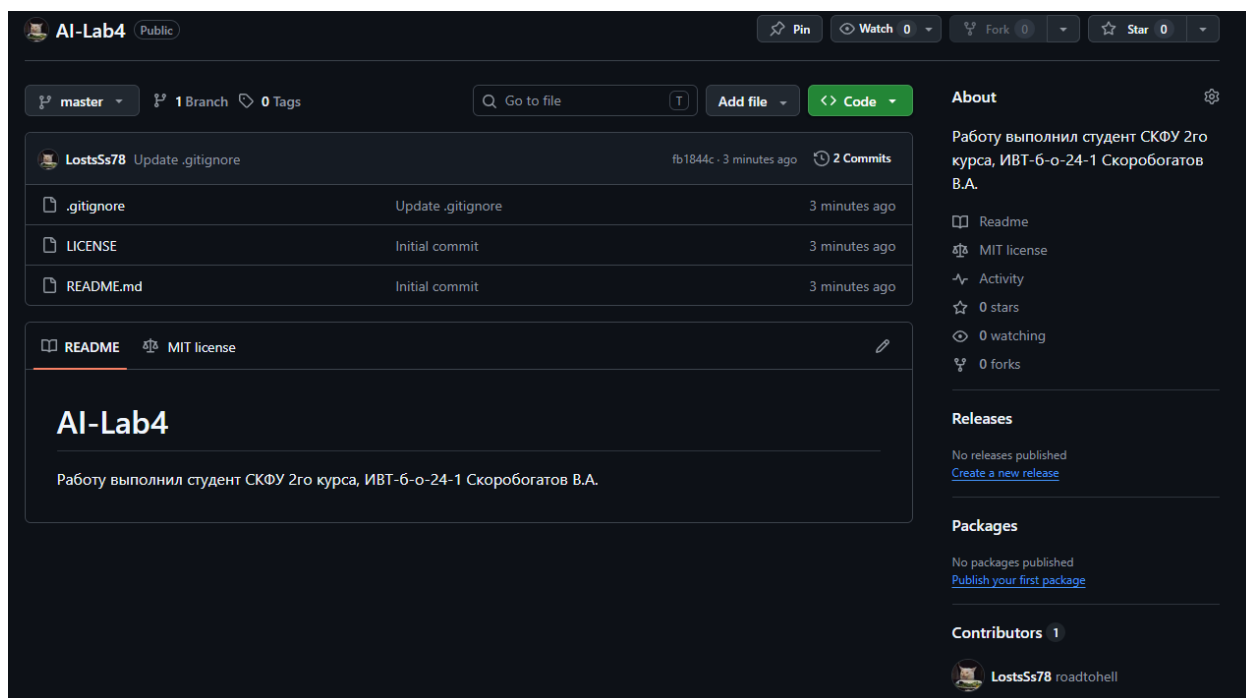


Рисунок 1. Репозиторий на GitHub

Далее были проработаны примеры, приведённые в лабораторной работе:

Пример 1

```
import pandas as pd

# Создадим Series с пользовательскими индексами
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

# Доступ к элементам по порядковому номеру
print(s.iloc[0]) # Первый элемент (10)
print(s.iloc[2]) # Третий элемент (30)
print(s.iloc[-1]) # Последний элемент (50)

10
30
50
```

Рисунок 2. Пример 1

Пример 2

```
# Срез с использованием iloc (позиционный индекс, невключительный stop)
print(s.iloc[1:3]) # Выведет элементы с индексами 1 и 2 (20, 30)

# Срез с использованием loc (меточная индексация, включительный stop)
print(s.loc['b':'d']) # Выведет элементы с индексами 'b', 'c' и 'd' (20, 30, 40)

b    20
c    30
dtype: int64
b    20
c    30
d    40
dtype: int64
```

Рисунок 3. Пример 2

Пример 3

```
# Создаём Series с числовыми значениями
s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

# Фильтруем значения больше 10
filtered_s = s[s > 10]

print(filtered_s)

b    25
d    30
e    15
dtype: int64
```

Рисунок 4. Пример 3

Пример 4

```
# Создаём Series
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

# Изменяем значение элемента с индексом 'b'
s.loc['b'] = 25
print(s)

a    10
b    25
c    30
d    40
dtype: int64
```

Рисунок 5. Пример 4

Пример 5

```
s_with_nan = pd.Series([10, None, 30, None], index=['a', 'b', 'c', 'd'])

# Заполняем пропущенные значения числом 0
s_filled = s_with_nan.fillna(0)
print(s_filled)

a    10.0
b     0.0
c    30.0
d     0.0
dtype: float64
```

Рисунок 6. Пример 5

Пример 6

```
# Создаём Series
s = pd.Series([10, 20, 30, 40, 50, 60, 70], index=['a', 'b', 'c', 'd', 'e', 'f', 'g'])

# Вывод первых 3 элементов
print(s.head(3))

a    10
b    20
c    30
dtype: int64
```

Рисунок 7. Пример 6

Пример 7

```
# Создаём Series с пользовательскими индексами
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

# Получение индексов
print(s.index)

Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

Рисунок 8. Пример 7

Пример 8

```
s1 = pd.Series([1.5, 2.3, 3.7]) # Числа с плавающей запятой
s2 = pd.Series(["apple", "banana", "cherry"]) # Строки
s3 = pd.Series([True, False, True]) # Булевы значения

print(s1.dtype) # float64
print(s2.dtype) # object (строки)
print(s3.dtype) # bool

float64
object
bool
```

Рисунок 9. Пример 8

Пример 9

```
import numpy as np

# Создаём Series с пропущенными значениями
s = pd.Series([10, np.nan, 30, None, 50])

# Проверяем наличие пропущенных значений
print(s.isnull())

0    False
1     True
2    False
3     True
4    False
dtype: bool
```

Рисунок 10. Пример 9

Пример 10

```
# Создаём Series с пропущенными значениями
s = pd.Series([10, np.nan, 30, None, 50])

# Заполняем пропущенные значения нулём
s_filled = s.fillna(0)
print(s_filled)
```

```
0    10.0
1     0.0
2    30.0
3     0.0
4    50.0
dtype: float64
```

Рисунок 11. Пример 10

Пример 11

```
# Создаём Series
s = pd.Series([10, 20, 30, 40, 50])

# Умножаем все элементы на 2
s_multiplied = s * 2
print(s_multiplied)
```

```
0     20
1     40
2     60
3     80
4    100
dtype: int64
```

Рисунок 12. Пример 11

Пример 12

```
s1 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s2 = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

# Поэлементное сложение двух Series
s_sum = s1 + s2
print(s_sum)
```

```
a     11
b     22
c     33
d     44
e     55
dtype: int64
```

Рисунок 13. Пример 12

Пример 13

```
s3 = pd.Series([100, 200, 300], index=['a', 'b', 'f'])  
  
# Сложение Series с несовпадающими индексами  
s_result = s1 + s3  
print(s_result)  
  
a    101.0  
b    202.0  
c      NaN  
d      NaN  
e      NaN  
f      NaN  
dtype: float64
```

Рисунок 14. Пример 13

Пример 14

```
# Создаём Series  
s = pd.Series([1, 2, 3, 4, 5])  
  
# Возводим в квадрат каждый элемент  
s_squared = s.apply(lambda x: x ** 2)  
print(s_squared)  
  
0    1  
1    4  
2    9  
3   16  
4   25  
dtype: int64
```

Рисунок 15. Пример 14

Пример 15

```
def custom_function(x):  
    return f"Value: {x}"  
  
s_transformed = s.apply(custom_function)  
print(s_transformed)  
  
0    Value: 1  
1    Value: 2  
2    Value: 3  
3    Value: 4  
4    Value: 5  
dtype: object
```

Рисунок 16. Пример 15

Пример 16

```
s_text = pd.Series(["apple", "banana", "cherry"])

# Преобразуем текст в верхний регистр
s_upper = s_text.apply(str.upper)
print(s_upper)
```

```
0    APPLE
1    BANANA
2    CHERRY
dtype: object
```

Рисунок 17. Пример 16

Пример 18

```
# Создаём Series
s = pd.Series([1, 2, 3, 4, 5])

# Применяем натуральный логарифм
s_log = np.log(s)
print(s_log)
```

```
0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
dtype: float64
```

Рисунок 18. Пример 17

Пример 18

```
# Создаём Series
s = pd.Series([1, 2, 3, 4, 5])

# Применяем натуральный логарифм
s_log = np.log(s)
print(s_log)
```

```
0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
dtype: float64
```

Рисунок 19. Пример 18

Пример 23

```
# Создаём список дат
dates = pd.date_range(start='2024-03-01', periods=5, freq='D')

# Создаём Series с временными метками в качестве индекса
s = pd.Series([100, 105, 102, 98, 110], index=dates)
print(s)
```

2024-03-01	100
2024-03-02	105
2024-03-03	102
2024-03-04	98
2024-03-05	110

Freq: D, dtype: int64

Рисунок 20. Пример 23

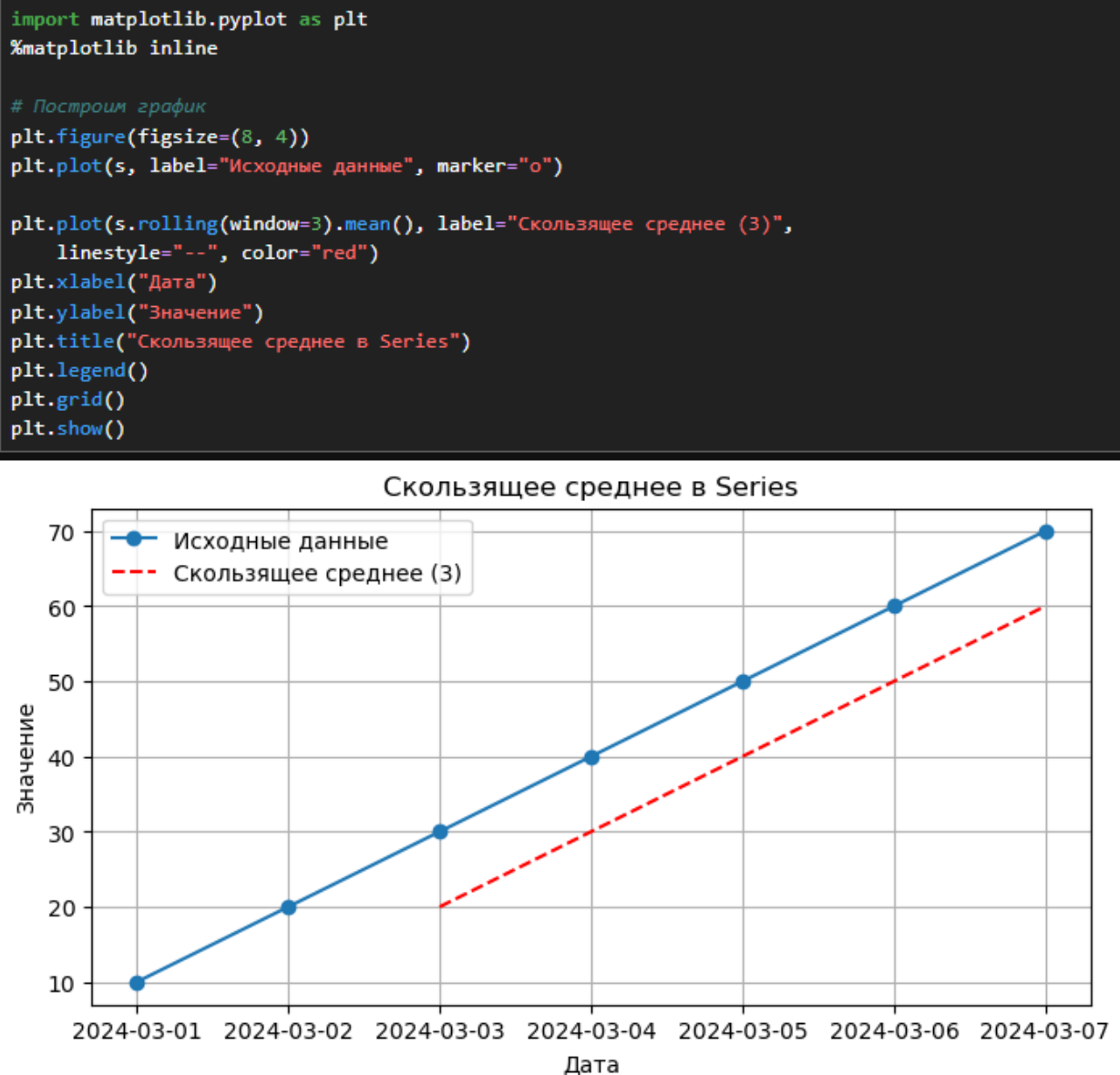


Рисунок 21. Пример 26

```
# Генерируем данные о цене акции
dates = pd.date_range(start="2024-01-01", periods=30, freq="D")
prices = pd.Series(np.random.randint(90, 110, size=30), index=dates)

# Вычисляем процентный прирост
returns = prices.pct_change() * 100

# Построим график
plt.figure(figsize=(10, 5))

plt.plot(prices, label="Цена акции", marker="o")
plt.xlabel("Цена")
plt.ylabel("Дата")
plt.title("График цены акции")

plt.figure(figsize=(10, 5))
plt.bar(returns.index, returns, color="red", alpha=0.7, label="Процентное изменение")
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Изменение (%)")
plt.ylabel("Дата")
plt.title("Процентное изменение цены акции")
plt.legend()
plt.show()
```

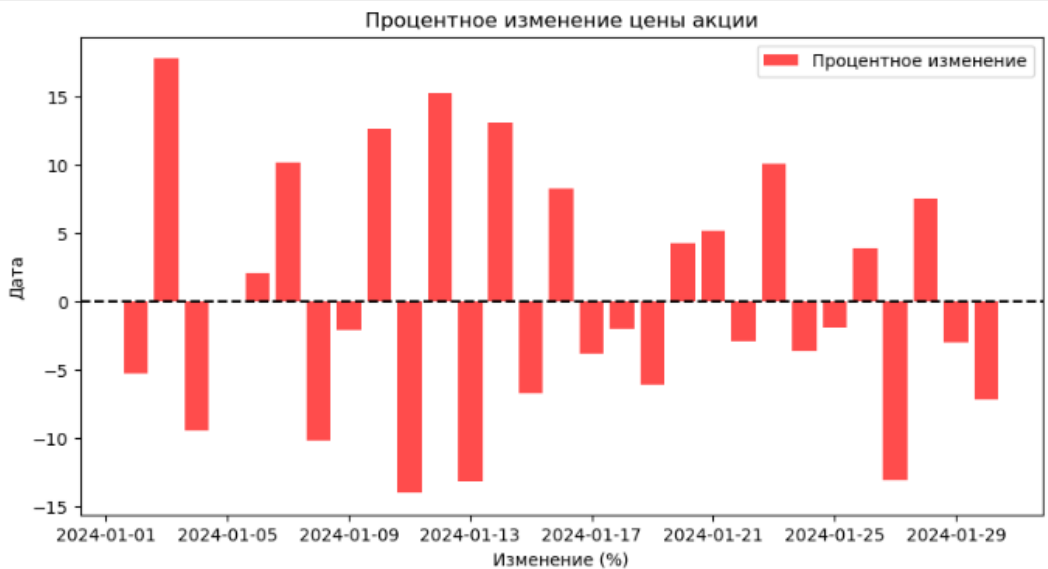
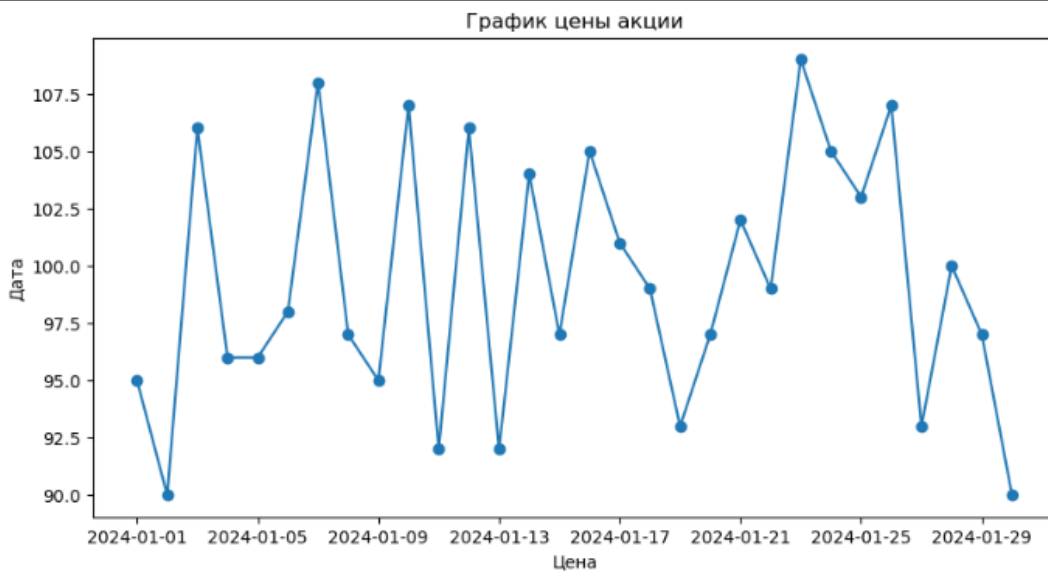


Рисунок 22. Пример 28

Далее были выполнены задания, представленные в лабораторной работе:

1. Создание Series из списка

Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e'].

Выведите его на экран и определите его тип данных.

```
import pandas as pd

values = [5, 15, 25, 35, 45]
indices = ["a", "b", "c", "d", "e"]

series = pd.Series(values, index=indices)

print(series)
print(type(series))
print(series.dtype)

a    5
b   15
c   25
d   35
e   45
dtype: int64
<class 'pandas.core.series.Series'>
int64
```

Рисунок 23. Задание 1

2. Получение элемента Series

Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60].

Используйте:

- `.loc[]` для получения элемента с индексом 'C';
- `.iloc[]` для получения третьего элемента.

```
indices = ["A", "B", "C", "D", "E"]
values = [12, 24, 36, 48, 60]

series = pd.Series(values, index=indices)

element_loc = series.loc["C"]
element_iloc = series.iloc[2]

print("Элемент с индексом 'C' (.loc):", element_loc)
print("Третий элемент (.iloc):", element_iloc)

Элемент с индексом 'C' (.loc): 36
Третий элемент (.iloc): 36
```

Рисунок 24. Задание 2

3. Фильтрация данных с помощью логической индексации

Создайте Series из массива NumPy:

```
np.array([4, 9, 16, 25, 36, 49, 64]).
```

Выберите только те элементы, которые больше 20, и выведите результат.

```
import numpy as np

array = np.array([4, 9, 16, 25, 36, 49, 64])
series = pd.Series(array)

filtered_series = series[series > 20]

print(filtered_series)
```

```
3    25
4    36
5    49
6    64
dtype: int32
```

Рисунок 25. Задание 3

4. Просмотр первых и последних элементов

Создайте Series, содержащий 50 случайных целых чисел от 1 до 100 (используйте np.random.randint).

Выведите первые 7 и последние 5 элементов с помощью методов .head() и .tail().

```
np.random.seed(42)
data = np.random.randint(1, 101, 50)

series = pd.Series(data)

print("Первые 7 элементов:")
print(series.head(7))

print("\nПоследние 5 элементов:")
print(series.tail(5))
```

```
Первые 7 элементов:
0    52
1    93
2    15
3    72
4    61
5    21
6    83
dtype: int32

Последние 5 элементов:
45    51
46     7
47    21
48    73
49    39
dtype: int32
```

Рисунок 26. Задание 4

5. Определение типа данных Series

Создайте Series из списка:

`['cat', 'dog', 'rabbit', 'parrot', 'fish']`.

Определите тип данных с помощью `.dtype`, затем преобразуйте его в `category` с помощью `.astype()`.

```
animals = ["cat", "dog", "rabbit", "parrot", "fish"]

series = pd.Series(animals)

print("Исходный тип данных:", series.dtype)

series = series.astype("category")

print("Тип данных после преобразования:", series.dtype)
```

Исходный тип данных: object

Тип данных после преобразования: category

Рисунок 27. Задание 5

6. Проверка пропущенных значений

Создайте Series с данными:

`[1.2, np.nan, 3.4, np.nan, 5.6, 6.8]`.

Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN), и выведите индексы таких элементов.

```
data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]

series = pd.Series(data)

has_nan = series.isna().any()
nan_indices = series[series.isna()].index

print("Есть ли пропущенные значения:", has_nan)
print("Индексы пропущенных значений:", list(nan_indices))
```

Есть ли пропущенные значения: True

Индексы пропущенных значений: [1, 3]

Рисунок 28. Задание 6

7. Заполнение пропущенных значений

Используйте Series из предыдущего задания и замените все NaN на среднее значение всех непустых элементов. Выведите результат.

```
data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
series = pd.Series(data)

mean_value = series.mean()

filled_series = series.fillna(mean_value)

print("Среднее значение:", mean_value)
print("Series после заполнения NaN:")
print(filled_series)
```

```
Среднее значение: 4.25
Series после заполнения NaN:
0    1.20
1    4.25
2    3.40
3    4.25
4    5.60
5    6.80
dtype: float64
```

Рисунок 29. Задание 7

8. Арифметические операции с Series

Создайте два Series:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
```

Выполните сложение $s1 + s2$.

Объясните, почему в результате появляются NaN, и замените их на 0.

```
s1 = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
s2 = pd.Series([5, 15, 25, 35], index=["b", "c", "d", "e"])

result = s1 + s2

print("Результат сложения:")
print(result)

# Замена NaN на 0
result_filled = result.fillna(0)

print("\nПосле замены NaN на 0:")
print(result_filled)
```

```
Результат сложения:
a      NaN
b    25.0
c    45.0
d    65.0
e      NaN
dtype: float64
```

```
После замены NaN на 0:
a     0.0
b    25.0
c    45.0
d    65.0
e     0.0
dtype: float64
```

Рисунок 30. Задание 8

9. Применение функции к Series

Создайте Series из чисел [2, 4, 6, 8, 10].

Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `.apply(np.sqrt)`.

```
numbers = pd.Series([2, 4, 6, 8, 10])
```

```
sqrt_series = numbers.apply(np.sqrt)
```

```
print("Исходный Series:")
```

```
print(numbers)
```

```
print("\nПосле применения sqrt:")
```

```
print(sqrt_series)
```

Исходный Series:

0 2

1 4

2 6

3 8

4 10

dtype: int64

После применения sqrt:

0 1.414214

1 2.000000

2 2.449490

3 2.828427

4 3.162278

dtype: float64

Рисунок 31. Задание 9

10. Основные статистические методы

Создайте Series из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`).

Найдите:

- сумму,
- среднее,
- минимальное значение,
- максимальное значение,
- стандартное отклонение.

```
np.random.seed(42)
data = np.random.randint(50, 151, 20)

series = pd.Series(data)

print("Series:")
print(series)

print("\nСумма:", series.sum())
print("Среднее:", series.mean())
print("Минимум:", series.min())
print("Максимум:", series.max())
print("Стандартное отклонение:", series.std())
```

```
Series:
0      101
1      142
2       64
3      121
4      110
5       70
6      132
7      136
8      124
9      124
10     137
11     149
12      73
13      52
```

```
Сумма: 2062
Среднее: 103.1
Минимум: 51
Максимум: 149
Стандартное отклонение: 32.28964311712935
```

Рисунок 32. Задание 10

II. Работа с временными рядами

Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года:

```
pd.date_range(start='2024-03-01', periods=10, freq='D')
```

Значения — случайные числа от 10 до 100.

Выберите данные за период с 5 по 8 марта.

```
np.random.seed(42)

dates = pd.date_range(start="2024-03-01", periods=10, freq="D")
values = np.random.randint(10, 101, 10)

series = pd.Series(values, index=dates)

print("Весь временной ряд:")
print(series)

filtered = series["2024-03-05":"2024-03-08"]

print("\nДанные с 5 по 8 марта:")
print(filtered)
```

Весь временной ряд:

2024-03-01	61
2024-03-02	24
2024-03-03	81
2024-03-04	70
2024-03-05	30
2024-03-06	92
2024-03-07	96
2024-03-08	84
2024-03-09	84
2024-03-10	97

Freq: D, dtype: int32

Данные с 5 по 8 марта:

2024-03-05	30
2024-03-06	92
2024-03-07	96
2024-03-08	84

Freq: D, dtype: int32

Рисунок 33. Задание 11

12. Проверка уникальности индексов

Создайте Series с индексами:

['A', 'B', 'A', 'C', 'D', 'B']

и значениями:

[10, 20, 30, 40, 50, 60]

Проверьте, являются ли индексы уникальными.

Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

```
indices = ["A", "B", "A", "C", "D", "B"]
values = [10, 20, 30, 40, 50, 60]

series = pd.Series(values, index=indices)

print("Series:")
print(series)

print("\nИндексы уникальны?:", series.index.is_unique)

if not series.index.is_unique:
    grouped = series.groupby(level=0).sum()
    print("\nПосле группировки и суммирования:")
    print(grouped)
```

```
Series:
A      10
B      20
A      30
C      40
D      50
B      60
dtype: int64

Индексы уникальны?: False

После группировки и суммирования:
A      40
B      80
C      40
D      50
dtype: int64
```

Рисунок 34. Задание 12

13. Преобразование строковых дат в DatetimeIndex

Создайте Series, где индексами будут строки:

['2024-03-10', '2024-03-11', '2024-03-12'],

а значениями:

[100, 200, 300].

Преобразуйте индексы в DatetimeIndex и выведите тип данных индекса.

```
dates = ["2024-03-10", "2024-03-11", "2024-03-12"]
values = [100, 200, 300]

series = pd.Series(values, index=dates)

print("До преобразования:")
print(series)
print("Тип индекса:", type(series.index))

# Преобразование индекса
series.index = pd.to_datetime(series.index)

print("\nПосле преобразования:")
print(series)
print("Тип индекса:", type(series.index))
```

```
До преобразования:
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
Тип индекса: <class 'pandas.core.indexes.base.Index'>

После преобразования:
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
Тип индекса: <class 'pandas.core.indexes.datetimes.DatetimeIndex'>
```

Рисунок 35. Задание 13

14. Чтение данных из CSV-файла

Создайте CSV-файл data.csv со следующими данными:

Дата,Цена 2024-03-01,100 2024-03-02,110 2024-03-03,105 2024-03-04,120 2024-03-05,115

Считайте файл с помощью pandas и выведите полученный DataFrame.

```
data = {  
    "Дата": [  
        "2024-03-01",  
        "2024-03-02",  
        "2024-03-03",  
        "2024-03-04",  
        "2024-03-05"  
    ],  
    "Цена": [100, 110, 105, 120, 115]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("data.csv", index=False)
```

```
print("Файл data.csv создан.")
```

Файл data.csv создан.

```
df = pd.read_csv("data.csv")
```

```
print("Считанный DataFrame:")
```

```
print(df)
```

Считанный DataFrame:

	Дата	Цена
0	2024-03-01	100
1	2024-03-02	110
2	2024-03-03	105
3	2024-03-04	120
4	2024-03-05	115

Рисунок 36. Задание 14

15. Построение графика на основе Series

Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года, а значениями — случайные числа от 50 до 150.

Постройте график значений с помощью matplotlib.

Добавьте:

- заголовок,
- подписи осей,
- сетку.

```
import matplotlib.pyplot as plt
%matplotlib inline

# Фиксация генератора случайных чисел
np.random.seed(42)

# Создание дат
dates = pd.date_range(start="2024-03-01", end="2024-03-30", freq="D")

# Генерация случайных значений
values = np.random.randint(50, 151, len(dates))

# Создание Series
series = pd.Series(values, index=dates)

# Построение графика
plt.figure(figsize=(10, 6))

plt.plot(series.index, series.values, marker="o")

plt.title("Случайные значения за март 2024 года")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.grid(True)

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

Рисунок 37. Задание 15

Также было выполнено индивидуальное задание:

1. Сравнение температуры в двух городах

Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года, а значениями — температуры в двух городах.

Постройте графики изменения температуры на одном графике.

Добавьте:

- легенду,
- выделите день с наибольшей разницей температур между городами.

```

# Фиксация генератора случайных чисел
np.random.seed(42)

# Создание дат
dates = pd.date_range(start="2024-03-01", end="2024-03-10", freq="D")

# Генерация температур
temp_city1 = np.random.randint(-5, 10, len(dates))
temp_city2 = np.random.randint(-5, 10, len(dates))

# Создание Series
city1 = pd.Series(temp_city1, index=dates)
city2 = pd.Series(temp_city2, index=dates)

# Вычисление разницы температур
difference = (city1 - city2).abs()

max_diff_date = difference.idxmax()
max_diff_value = difference.max()

# Построение графика
plt.figure(figsize=(10, 6))

plt.plot(city1.index, city1.values, marker="o", label="Город 1")
plt.plot(city2.index, city2.values, marker="s", label="Город 2")

# Выделение дня с максимальной разницей
plt.axvline(max_diff_date, color="red", linestyle="--",
            label="Максимальная разница")

plt.scatter(
    max_diff_date,
    city1[max_diff_date],
    color="red",
    zorder=5
)

plt.scatter(
    max_diff_date,
    city2[max_diff_date],
    color="red",
    zorder=5
)

```

Рисунок 38. Код индивидуального задания

```
plt.title("Сравнение температур в двух городах")
plt.xlabel("Дата")
plt.ylabel("Температура (°C)")
plt.legend()
plt.grid(True)

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()

print("Дата с наибольшей разницей:", max_diff_date.date())
print("Максимальная разница температур:", max_diff_value)
```

Рисунок 39. Код индивидуального задания

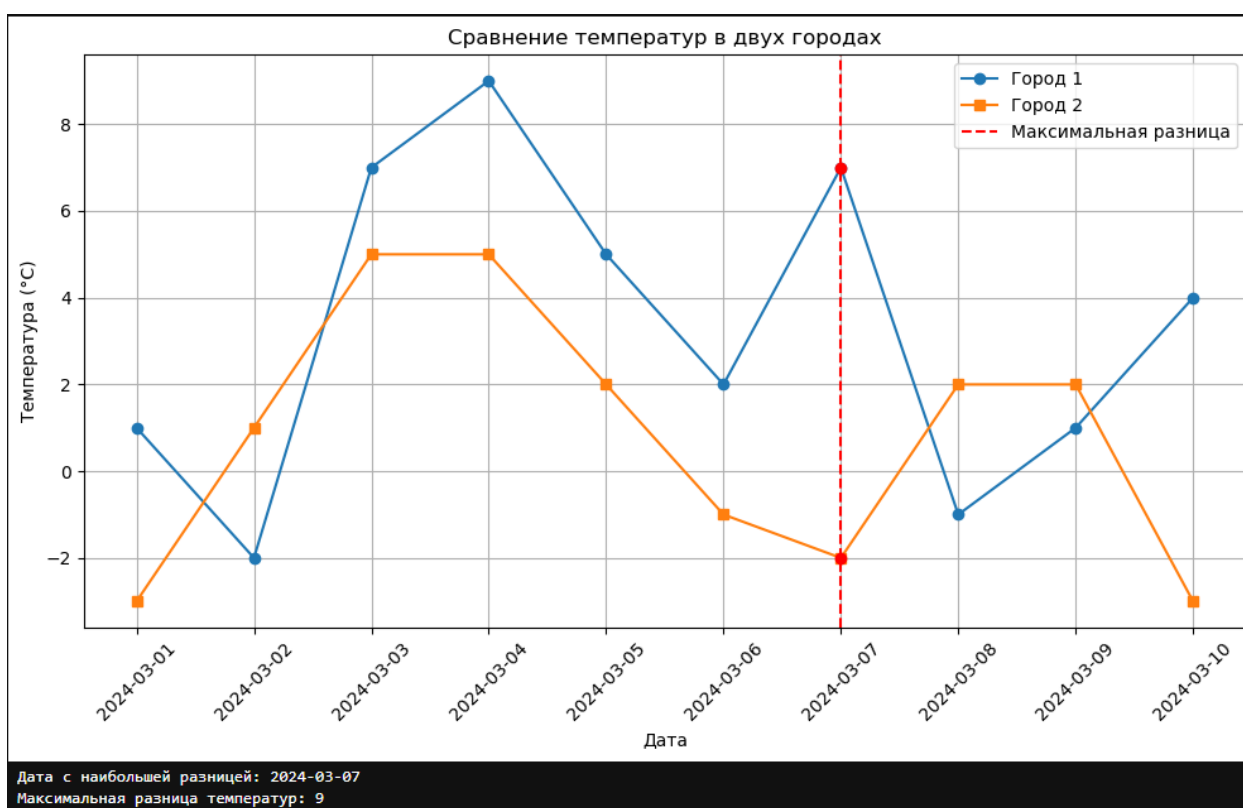


Рисунок 40. Результат выполнения кода индивидуального задания

Ответы на контрольные вопросы:

1. Что такое `pandas.Series` и чем она отличается от списка в Python?

`pandas.Series` - это одномерная структура данных библиотеки `pandas`, похожая на массив или столбец таблицы. Главное отличие от обычного списка Python состоит в том, что `Series` имеет индекс, то есть метки элементов, поддерживает быстрые векторизованные операции, хранит тип данных элементов, умеет работать с пропущенными значениями и

автоматически выравнивает данные по индексам при операциях. Список Python является более простой структурой и не имеет встроенных средств анализа данных.

2. Какие типы данных можно использовать для создания Series?

Для создания Series можно использовать списки, кортежи, массивы NumPy, словари, скалярные значения, диапазоны, другие Series, объекты Index, а также данные, загруженные из файлов. Элементы Series могут иметь разные типы: целые числа, вещественные числа, строки, логические значения, даты и время, временные интервалы, категориальные данные, комплексные числа и объектный тип object.

3. Как задать индексы при создании Series?

Индексы задаются параметром index при создании объекта Series. Количество индексов должно соответствовать количеству элементов данных. Если Series создается из словаря, то ключи словаря по умолчанию становятся индексами.

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)
```

4. Каким образом можно обратиться к элементу Series по его индексу?

К элементу Series можно обратиться по метке индекса через квадратные скобки или через метод .loc[]. Если необходимо обратиться к элементу по порядковому номеру, используется .iloc[].

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

print(s['b'])          # обращение по метке индекса
print(s.loc['b'])       # обращение по метке через loc
print(s.iloc[1])        # обращение по позиции
```

5. В чём разница между .iloc[] и .loc[] при индексации Series?

`.loc[]` выполняет обращение по меткам индекса, а `.iloc[]` - по целочисленным позициям элементов. Например, `s.loc['a']` выбирает элемент с индексом 'a', а `s.iloc[0]` выбирает первый элемент Series. Важно учитывать, что если индекс состоит из чисел, то `s.loc[0]` обращается к метке 0, а `s.iloc[0]` — к первому элементу по порядку. При срезах `.loc[]` включает правую границу для меток, а `.iloc[]` работает как обычные срезы Python и не включает правую границу.

6. Как использовать логическую индексацию в Series?

Логическая индексация используется для выбора элементов, удовлетворяющих условию. Сначала создается логическая маска, содержащая значения True и False, затем эта маска передается в квадратные скобки. Для объединения условий используются операторы `&`: логическое «и», `|`: логическое «или», `~`: отрицание. Каждое условие рекомендуется заключать в круглые скобки.

```
s = pd.Series([5, 12, 18, 3, 25])

result = s[s > 10]
result2 = s[(s > 10) & (s < 20)]
print(result)
```

7. Какие методы можно использовать для просмотра первых и последних элементов Series?

Для просмотра первых элементов Series используется метод `.head()`, а для просмотра последних - метод `.tail()`. По умолчанию оба метода выводят по 5 элементов, но можно передать нужное количество в качестве аргумента: `s.head(10)` или `s.tail(3)`.

8. Как проверить тип данных элементов Series?

Тип данных элементов Series можно проверить с помощью атрибута `.dtype`. Также можно использовать `.dtypes`, а для просмотра общей информации о Series - метод `.info()`. Если необходимо узнать фактические

типы отдельных объектов в Series с типом object, можно применить `s.apply(type)`.

```
print(s.dtype)
print(s.apply(type).value_counts())
```

9. Каким способом можно изменить тип данных Series?

Тип данных Series изменяется методом `.astype()`. Например, `s.astype(float)` преобразует элементы в вещественный тип. Для специальных преобразований используются функции pandas: `pd.to_numeric()` - преобразование к числам, `pd.to_datetime()` - преобразование к датам, `pd.to_timedelta()` - преобразование к временным интервалам. Параметр `errors='coerce'` позволяет заменить некорректные значения на NaN или NaT.

```
s = pd.Series(['1', '2', '3'])
s = s.astype(int)

dates = pd.to_datetime(pd.Series(['2026-01-01', '2026-01-02']))
```

10. Как проверить наличие пропущенных значений в Series?

Наличие пропущенных значений проверяется методами `.isna()` или `.isnull()`, которые возвращают логическую маску. Чтобы определить, есть ли хотя бы одно пропущенное значение, используют `s.isna().any()`. Чтобы посчитать количество пропусков, используют `s.isna().sum()`. Обратная проверка выполняется методами `.notna()` или `.notnull()`.

```
print(s.isna())
print(s.isna().any())
print(s.isna().sum())
```

11. Какие методы используются для заполнения пропущенных значений в Series?

Для заполнения пропущенных значений используется метод `.fillna()`. Пропуски можно заменить константой, средним значением, медианой, модой, предыдущим или следующим известным значением. Для временных рядов также часто используется интерполяция методом `.interpolate()`.

```
s1 = s.fillna(0)
s2 = s.fillna(s.mean())
s3 = s.ffill()    # заполнение предыдущим значением
s4 = s.bfill()    # заполнение следующим значением
s5 = s.interpolate()
```

12. Чем отличается метод `.fillna()` от `.dropna()`?

Метод `.fillna()` не удаляет пропущенные значения, а заменяет их указанным значением или рассчитанной величиной. Метод `.dropna()` удаляет элементы `Series`, в которых содержатся пропущенные значения. Таким образом, `.fillna()` сохраняет размер `Series`, а `.dropna()` уменьшает количество элементов. Выбор метода зависит от задачи: если пропуски можно корректно заменить, используют `.fillna()`, а если такие элементы лучше исключить из анализа, используют `.dropna()`.

13. Какие математические операции можно выполнять с `Series`?

С `Series` можно выполнять арифметические операции: сложение, вычитание, умножение, деление, целочисленное деление, возведение в степень и взятие остатка. Эти операции могут выполняться со скалярами или с другими `Series`. Также доступны математические функции NumPy, например `np.sqrt()`, `np.log()`, `np.sin()`. При операциях между двумя `Series` pandas выравнивает данные по индексам, а если индекс отсутствует в одной из `Series`, результатом может стать `NaN`.

14. В чём преимущество векторизованных операций по сравнению с циклами Python?

Векторизованные операции выполняются сразу над всем массивом данных и обычно реализованы на оптимизированном низкоуровневом коде. Поэтому они работают быстрее обычных циклов Python, код получается короче и понятнее, а вероятность ошибок снижается. Например, выражение `s * 2` умножает все элементы `Series` на 2 без явного цикла `for`.

15. Как применить пользовательскую функцию к каждому элементу Series?

Пользовательскую функцию к каждому элементу Series можно применить методом `.apply()` или `.map()`. Метод `.apply()` удобен для произвольных функций, а `.map()` часто используется для замены значений по словарю или простого поэлементного преобразования. Если возможно, предпочтительнее использовать векторизованные операции, так как они обычно быстрее.

```
s = pd.Series([1, 2, 3])

def square(x):
    return x ** 2

result = s.apply(square)
print(result)
```

16. Какие агрегирующие функции доступны в Series?

В Series доступны агрегирующие функции: `.sum()`, `.mean()`, `.median()`, `.min()`, `.max()`, `.std()`, `.var()`, `.count()`, `.size`, `.prod()`, `.quantile()`, `.mode()`, `.describe()`, `.nunique()`, `.idxmin()` и `.idxmax()`. Они позволяют получать обобщенные характеристики данных: сумму, среднее, медиану, разброс, количество значений и другие статистики.

17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series?

Минимальное значение находится методом `.min()`, максимальное - `.max()`, среднее - `.mean()`, стандартное отклонение - `.std()`. Все эти методы по умолчанию пропускают NaN-значения.

```
print(s.min())
print(s.max())
print(s.mean())
print(s.std())
```

18. Как сортировать Series по значениям и по индексам?

Для сортировки Series по значениям используется метод `.sort_values()`, а для сортировки по индексам - метод `.sort_index()`. По умолчанию сортировка выполняется по возрастанию. Для сортировки по убыванию нужно указать `ascending=False`.

```
s_sorted_values = s.sort_values()
s_sorted_index = s.sort_index()
s_desc = s.sort_values(ascending=False)
```

19. Как проверить, являются ли индексы Series уникальными?

Уникальность индексов Series проверяется атрибутом `s.index.is_unique`. Он возвращает `True`, если все индексы уникальны, и `False`, если есть повторяющиеся метки. Для поиска повторов можно использовать `s.index.duplicated()`.

```
print(s.index.is_unique)
print(s.index.duplicated())
```

20. Как сбросить индексы Series и сделать их числовыми?

Чтобы сбросить индексы Series и получить стандартную числовую нумерацию, используется метод `.reset_index()`. Если нужно отбросить старый индекс и оставить только значения, применяется параметр `drop=True`. В результате обычно получается Series с индексами 0, 1, 2 и так далее.

```
s_new = s.reset_index(drop=True)
```

21. Как можно задать новый индекс в Series?

Новый индекс можно задать при создании Series через параметр `index` или изменить позже присваиванием в атрибут `.index`. Также можно использовать метод `.set_axis()`. Количество новых индексов должно совпадать с количеством элементов Series.

```
s = pd.Series([10, 20, 30])
s.index = ['a', 'b', 'c']

s2 = s.set_axis(['x', 'y', 'z'])
```

22. Как работать с временными рядами в Series?

Для работы с временными рядами в Series в качестве индекса используют DatetimeIndex. Это позволяет выбирать данные по датам, выполнять срезы по временным периодам, изменять частоту данных, рассчитывать скользящие средние, группировать данные по месяцам, кварталам или годам и выполнять ресемплирование методом .resample().

```
dates = pd.date_range('2026-01-01', periods=5, freq='D')
s = pd.Series([10, 12, 15, 14, 18], index=dates)

print(s['2026-01'])
print(s.resample('M').mean())
```

23. Как преобразовать строковые даты в формат DatetimeIndex?

Строковые даты преобразуются в формат даты и времени с помощью функции pd.to_datetime(). Если нужно сделать даты индексом Series, результат преобразования можно передать в параметр index или присвоить атрибуту .index. При наличии ошибок преобразования удобно использовать параметр errors='coerce', который заменит некорректные даты на NaT.

```
dates = pd.to_datetime(['2026-01-01', '2026-01-02', '2026-01-03'])
s = pd.Series([10, 20, 30], index=dates)
print(s.index)
```

24. Каким образом можно выбрать данные за определённый временной диапазон?

Если Series имеет индекс DatetimeIndex, данные за определенный временной диапазон можно выбрать с помощью среза по строковым датам через .loc[]. Например, s.loc['2026-01-01':'2026-01-31'] выберет данные за январь 2026 года. Также можно выбирать данные по году, месяцу или конкретной дате, если индекс имеет временной тип.

```
january = s.loc['2026-01-01':'2026-01-31']
year_data = s.loc['2026']
month_data = s.loc['2026-01']
```

25. Как загрузить данные из CSV-файла в Series?

Данные из CSV-файла обычно сначала загружаются в DataFrame с помощью `pd.read_csv()`, а затем нужный столбец выбирается как Series. Если в файле один столбец, его можно сразу выбрать после чтения. Также можно использовать параметр `squeeze` в старых версиях pandas, но в современных версиях надежнее явно выбирать столбец.

```
df = pd.read_csv('data.csv')
s = df['column_name']
```

26. Как установить один из столбцов CSV-файла в качестве индекса Series?

Один из столбцов CSV-файла можно установить в качестве индекса при чтении файла с помощью параметра `index_col`. После этого нужный столбец DataFrame выбирается как Series. Если индекс содержит даты, можно дополнительно использовать `parse_dates`.

```
df = pd.read_csv('data.csv', index_col='date',
parse_dates=['date'])
s = df['value']
```

27. Для чего используется метод `.rolling().mean()` в Series?

Метод `.rolling().mean()` используется для расчета скользящего среднего. Он создает окно заданного размера и вычисляет среднее значение внутри этого окна для каждой позиции Series. Скользящее среднее помогает сглаживать случайные колебания, выявлять общий тренд и анализировать временные ряды.

```
s_rolling = s.rolling(window=3).mean()
```

28. Как работает метод `.pct_change()`? Какие задачи он решает?

Метод `.pct_change()` вычисляет относительное процентное изменение текущего значения по сравнению с предыдущим. Формула имеет вид: $(\text{текущее значение} - \text{предыдущее значение}) / \text{предыдущее значение}$. Метод используется для анализа темпов роста, доходности, изменения цен, динамики показателей во времени и сравнения относительных изменений.

```
changes = s.pct_change()  
changes_percent = s.pct_change() * 100
```

29. В каких ситуациях полезно использовать .rolling() и .pct_change()?

.rolling() полезен при анализе временных рядов, когда нужно рассчитать скользящее среднее, скользящую сумму, скользящее стандартное отклонение или другие показатели в движущемся окне. Это помогает сгладить данные и увидеть тренд. Метод .pct_change() полезен, когда нужно оценить относительное изменение показателя между периодами, например рост продаж, изменение курса валюты, доходность акции или изменение температуры относительно предыдущего дня. Вместе эти методы позволяют анализировать динамику данных и выявлять тенденции.

30. Почему NaN могут появляться в Series, и как с ними работать?

NaN в Series могут появляться из-за отсутствующих данных в исходном файле, ошибок преобразования типов, деления на ноль, объединения данных с несовпадающими индексами, результатов операций .rolling() и .pct_change(), а также из-за явного добавления пропусков. Работать с NaN можно несколькими способами: обнаруживать их методами .isna() и .isnull(), подсчитывать через .sum(), удалять методом .dropna(), заполнять методом .fillna(), интерполировать методом .interpolate() или заменять значениями, рассчитанными по данным, например средним, медианой или предыдущим значением.

Вывод: в ходе работы были получены знания по работе с основами библиотеки pandas, в частности, со структурой данных Series.